

server-state 개요

`server-state` 는 FARM/LAB GPU 서버의 공통 운영 기준을 정의하고, 각 서버가 해당 기준에 맞게 설정되어 있는지 점검하는 모듈이다. 구성요소별 설정 작업은 신규 서버를 같은 운영 기준에 맞게 구성할 때도 사용한다.

서버 상태는 OS 공통 패키지, Docker, NVIDIA, Kubernetes, storage network, Kerberos/NFS와 monitoring으로 나뉜다. 각 구성요소에는 목표 상태, 점검 방법, 설정 방법, 실행 형태와 안전 수준이 연결되어 있다.

문서	내용
설계	목표, 구조, 구성요소별 상태·점검·설정, 실행 형태와 안전 수준, 설정값과 코드 위치
운영	서버 등록, 환경 설정 변경, 구성요소 확장, 실행 제어 변경, 점검·설정과 검증 절차

server-state 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 링크를 누르면 GitHub 로그인 화면을 거쳐 해당 파일로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`server-state` 의 주요 목표는 FARM/LAB GPU 서버가 공통 운영 기준에 맞게 설정되어 있는지 일관된 방법으로 점검하는 것이다. 운영 기준은 OS 공통 패키지와 Docker, NVIDIA driver/runtime, Kubernetes, storage network, Kerberos/NFS, monitoring의 목표 상태와 점검 순서를 정의한다.

구성요소(component)는 운영 기준의 단위로, 독립적으로 점검하고 설정할 수 있다. 예를 들어 `docker-engine` 은 Docker 설치, service, daemon과 cgroup driver 상태를 하나의 구성요소로 관리한다.

`server-state` 는 이미 등록된 서버가 기준에서 벗어났는지 점검하고 교정하는 데에도, 신규 서버를 처음부터 같은 기준에 맞게 구성하는 데에도 쓸 수 있다. 두 경우 모두 정책에 정의된 같은 구성요소 순서를 그대로 따른다.

2. 설계 구조

Ansible playbook은 서버에서 실행할 작업 순서를 YAML로 정의한 파일이다. `server-state` 는 구성요소별 점검·설정을 이 playbook으로 수행한다.

구분	저장 위치	역할
정책	<code>policy/standard-gpu-server.yml</code>	구성요소와 실행 순서를 기록한다.
구성요소 정의	<code>components/*.yml</code>	목표 상태, 점검 방법, 설정 방법과 승인 수준을 기록한다.
서버·환경 정보	<code>servers.jsonl</code> , <code>config/environments.yml</code>	서버 접속·network 정보와 FARM/LAB별 설정값을 제공한다.
명령 구성	<code>server_state/</code>	대상 서버에 맞는 playbook, tag와 변수를 조합한다.
Ansible 작업	<code>ansible/roles/</code> 와 playbook	구성요소별 점검과 서버 설정을 수행한다.
실행 파일	<code>bin/server-state</code>	<code>describe</code> , <code>audit</code> , <code>plan</code> , <code>apply</code> 명령을 시작한다.

관리자가 `bin/server-state` 로 `describe`, `audit`, `plan`, `apply` 중 하나를 실행하면, 각 구성요소들은 다음 순서로 함께 동작한다. 네 가지 명령 모두 순서를 공통으로 따르며, 명령별로 달라지는 세부 동작은 4절에서 설명한다.

1. `--hosts` 로 대상 서버를 선택한다.
2. `--component` 로 구성요소를 선택한다.
3. 서버·환경 정보(`servers.jsonl`, `config/environments.yml`)와 FARM/LAB 설정을 결합해 실행값을 만든다.
4. 정책과 구성요소 정의에서 해당 구성요소에 연결된 `audit` 또는 `converge` playbook을 선택한다.
5. Ansible 명령을 화면에 보여주거나 실행한다.

3. 구성요소

`standard-gpu-server.yml` 은 10개 구성요소의 실행 순서를 정의한다. 모든 구성요소는 `server-state` 정책과 CLI에서 관리한다. 각 구성요소는 서버의 현재 상태를 점검하는 작업과, 서버를 목표 상태에 맞게 설정하는 작업을 정의한다.

각 구성요소는 아래 네 항목으로 설명된다.

항목	의미
목표 상태	서버가 도달해야 하는 상태
점검 (<code>audit</code>)	목표 상태 충족 여부를 확인하는 방법. 서버 상태를 읽기만 하고 아무것도 변경하지 않는다.
설정 (<code>converge</code>)	서버를 목표 상태로 만드는 방법. 서버를 실제로 바꿔서 목표 상태로 만든다.
설정 실행	설정 작업의 실행 형태와 안전 수준

실행 형태는 설정 작업을 어떻게 수행하는지, 안전 수준은 실행 전 어떤 확인·승인이 필요한지를 나타낸다. 두 값이 실제 명령에서 어떻게 쓰이는지는 4절에서 설명한다.

실행 형태	동작
<code>ansible-playbook</code>	설정 작업을 Ansible playbook으로 자동 실행한다.
<code>manual</code>	CLI가 수행할 절차를 안내하고, 관리자가 그 절차를 직접 수행한다.

안전 수준	실행 조건
safe	별도 확인 없이 바로 실행할 수 있다.
gated	운영 영향을 확인한 뒤 관리자 승인이 있어야 실행된다.
risky	작업 영향과 운영 일정까지 확인한 뒤 관리자 승인이 있어야 실행된다.

3.1 baseline-access

목표 상태: 서버가 inventory(`servers.jsonl`)와 Ansible inventory(`monitoring/ansible_playbook/inventory.ini`)에 등록되어 있고, 설정된 hostname으로 SSH 접속과 비대화형 sudo를 사용할 수 있다.

점검: Ansible ping으로 접속을 확인하고, `sudo -n true` 실행과 실제 hostname-inventory hostname 일치 여부를 확인한다.

설정: IP, hostname, SSH key, 관리 계정과 sudo 권한을 준비하고 두 inventory에 서버를 등록한다. 등록 절차는 [운영 문서](#)의 "2. 신규 서버 추가"를 참고한다.

설정 실행: 실행 형태 `manual`, 안전 수준 `gated`. 관리자가 준비 절차를 수행한다. 이 구성요소는 이후 Ansible 작업을 실행하기 위한 선행 조건이다.

관련 코드: `baseline-access.yml`, `baseline_access/tasks/audit.yml`

3.2 os-common

목표 상태: 서버가 정책에 지정된 Ubuntu release를 사용하고, 다른 구성요소에 필요한 공통 package가 설치되어 있다. 주요 package는 `curl`, `gnupg`, `ethtool`, `krb5-user`, `nfs-common`, `keyutils`, `adcli` 와 `python3` 이다.

점검: OS 종류와 version을 확인하고 package facts에서 필수 package를 하나씩 확인한다.

설정: apt cache를 갱신하고 공통 package 목록을 설치한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `os-common.yml`, `os_common/tasks/audit.yml`, `os_common/tasks/main.yml`

3.3 docker-engine

목표 상태: Docker Engine package가 설치되고 service가 활성화·실행되며, daemon이 응답하고 `systemd` cgroup driver를 사용한다. `daemon.json` 에는 `overlay2`, `json-file` 과 log size 설정이 포함된다.

점검: Docker service의 enabled·active 상태, `docker info` 응답과 cgroup driver 값을 확인한다.

설정: Docker apt key와 repository를 등록하고 Engine package를 설치한다. 기존 `daemon.json` 에 필요한 값을 병합하고 변경된 경우 Docker service를 재시작한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. package source와 daemon 설정이 변경되고 service가 재시작될 수 있어 운영 영향 확인이 필요하다.

관련 코드: `docker-engine.yml`, `docker_engine/tasks/audit.yml`, `docker_engine/tasks/main.yml`

3.4 nvidia-driver

목표 상태: 정책에 지정된 NVIDIA driver package가 설치되고 GPU를 정상 인식하며, 해당 package가 apt hold 상태로 유지된다.

점검: `nvidia-smi` 로 GPU 이름과 driver version을 읽고 `apt-mark showhold` 에서 NVIDIA driver package를 확인한다.

설정: 지정 driver package를 설치하고 apt hold를 설정한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `risky`. driver 변경은 GPU workload와 reboot 일정에 영향을 줄 수 있어 작업 영향과 운영 일정을 확인해야 한다.

관련 코드: `nvidia-driver.yml`, `nvidia_driver/tasks/audit.yml`, `nvidia_driver/tasks/main.yml`

3.5 nvidia-runtime

목표 상태: NVIDIA Container Toolkit이 설치되고 Docker와 containerd가 NVIDIA runtime을 사용할 수 있다.

점검: `nvidia-ctk` 설치, Docker runtime 목록과 containerd 설정의 NVIDIA runtime 항목을 확인한다.

설정: NVIDIA repository와 toolkit package를 설치하고 `nvidia-ctk runtime configure` 를 Docker와 containerd에 실행한다. 설정 변경 후 관련 service를 재시작한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. container runtime 설정이 변경되고 관련 service가 재시작될 수 있어 운영 영향 확인이 필요하다.

관련 코드: `nvidia-runtime.yml`, `nvidia_runtime/tasks/audit.yml`, `nvidia_runtime/tasks/main.yml`

3.6 kubernetes-packages

목표 상태: 정책에 지정된 version의 `kubeadm`, `kubelet`, `kubectrl` package가 설치·hold되고 kubelet service가 활성화되어 있다.

점검: 세 package의 설치 여부와 kubelet enabled 상태를 확인한다.

설정: Kubernetes apt key와 repository를 등록하고 세 package를 설치·hold한 뒤 kubelet을 활성화한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. Kubernetes package version이 바뀌고 kubelet에 영향을 줄 수 있어 운영 영향 확인이 필요하다.

관련 코드: `kubernetes-packages.yml`, `kubernetes_packages/tasks/audit.yml`, `kubernetes_packages/tasks/main.yml`

3.7 kubernetes-membership

목표 상태: 서버가 FARM 또는 LAB의 지정 Kubernetes cluster에 join되고, node에 해당 domain label이 설정되어 있다.

점검: 서버의 kubelet credential 파일을 확인하고 controller에서 node와 domain label을 조회한다.

설정: 관리자가 cluster, join token과 node 정보를 확인하고 검토를 마친 뒤 `kubeadm join` 명령을 실행한다.

설정 실행: 실행 형태 `manual`, 안전 수준 `risky`. join token의 유효 시간과 cluster 선택을 관리자가 확인한다.

관련 코드: `kubernetes-membership.yml`, `kubernetes_membership/tasks/audit.yml`,
`kubernetes_membership/tasks/main.yml`

3.8 storage-network

목표 상태: `servers.jsonl`에 storage interface가 기록되고, 해당 NIC의 RX queue를 4096으로 설정하는 systemd service가 활성화되어 있다.

점검: storage interface 값, 현재 RX queue 크기와 `decs-rx-queue.service` enabled 상태를 확인한다.

설정: RX queue를 설정하는 oneshot systemd unit을 설치하고 활성화한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `storage-network.yml`, `storage_network/tasks/audit.yml`, `storage_network/tasks/main.yml`

3.9 kerberos-nfs

목표 상태: 서버가 FARM/LAB Kerberos 설정과 host keytab을 사용해 machine principal 인증과 NFS service ticket 발급에 성공한다. `rpc-gssd`가 실행되고 공유 경로가 지정 source와 `sec=krb5` option으로 mount된다.

점검: `/etc/krb5.conf`, keytab, `kinit -k`, `kvno`, `rpc-gssd`와 mount source·option을 순서대로 확인한다. 자세한 Kerberos/NFS 인증·mount 구조는 [Kerberos/NFS 설계](#) 참고.

설정: domain Kerberos 설정을 설치하고 keytab과 principal을 검증한다. NFS GSS readiness·recovery unit을 설치하고 fstab에 mount를 기록한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `gated`. Kerberos 설정, fstab과 mount recovery에 영향을 줄 수 있어 운영 영향 확인이 필요하다. 실제 mount 실행은 별도 `server_state_mount_now` 값으로 제어한다.

관련 코드: `kerberos-nfs.yml`, `audit_client.yml`, `kerberos_nfs_client/`

3.10 monitoring

목표 상태: `cluster-monitor-exporter`와 `gpu-user-exporter` service가 활성화·실행되고, 두 metrics endpoint와 public health endpoint가 HTTP 200으로 응답한다.

점검: 두 exporter service의 enabled·active 상태와 로컬 metrics·health endpoint 응답을 확인한다.

설정: exporter binary를 build하고 설정 파일과 systemd unit을 설치한다. service를 시작한 뒤 metrics 응답을 검증한다.

설정 실행: 실행 형태 `ansible-playbook`, 안전 수준 `safe`.

관련 코드: `monitoring.yml`, `audit_exporters.yml`, `deploy_exporters.yml`

4. 명령별 동작

4.1 describe

`describe` 는 서버에 접속하지 않고, 정책과 구성요소 정의에 기록된 목표 상태, 점검·설정의 실행 형태와 안전 수준을 보여준다. `--component` 를 생략하면 정책에 포함된 전체 구성요소를 대상으로 한다.

4.2 audit

`audit` 은 구성요소의 읽기 전용 점검 playbook을 실행한다. 각 실행 결과는 서버와 구성요소 단위로 `OK` 또는 `FAILED` 로 표시된다. `--show-command` 는 같은 playbook과 변수를 사용한 명령을 화면에 출력한다.

4.3 plan

`plan` 은 구성요소의 설정 playbook을 Ansible `--check --diff` 로 실행한다. `package`, 파일과 `service`의 예상 변경을 확인할 수 있다. `manual` 구성요소는 실행 명령 대신 운영 절차 reference를 표시한다.

4.4 apply

`apply` 는 모든 선택 항목의 실행 형태와 안전 수준을 먼저 확인한다. `ansible-playbook` 구성요소는 `--execute` 로 실행하며, `gated` 는 `--approve-gated` , `risky` 는 `--approve-risky` 승인을 함께 사용한다. `manual` 구성요소는 운영 절차 reference를 표시한다.

5. 설정 구조

5.1 정책과 구성요소 정의

정책 파일은 구성요소 ID와 순서를 기록한다. 각 `components/<id>.yaml` 은 다음 정보를 기록한다.

필드	의미
<code>id</code>	<code>--component</code> 에서 사용하는 이름
<code>desired_state</code>	점검과 설정의 기준이 되는 서버 상태
<code>audit</code>	점검 playbook, tag와 <code>safe</code> 수준
<code>converge</code>	설정 playbook 또는 수동 절차와 안전 수준

`docker-engine.yaml` 은 다음과 같이 구성된다.

```

id: docker-engine
desired_state: Docker Engine is installed, enabled, responsive, and configured with the systemd cgroup driver.
audit:
  kind: ansible-playbook
  playbook: server-state/ansible/playbooks/audit.yml
  tags: [docker-engine]
  safety: safe
converge:
  kind: ansible-playbook
  playbook: server-state/ansible/playbooks/converge.yml
  tags: [docker-engine, cgroups]
  safety: gated

```

`audit` 명령은 audit playbook에 `docker-engine` tag를 전달한다. `plan` 과 `apply` 는 converge playbook에 `docker-engine,cgroups` tag를 전달한다. 실제 점검 task는 role의 `tasks/audit.yml`, 설정 task는 `tasks/main.yml`에 있다.

5.2 서버와 FARM/LAB 설정

`servers.jsonl` 은 host, server ID, domain, SSH 주소·port·계정과 management/storage interface 정보를 제공한다. `config/environments.yml` 은 FARM/LAB별 realm, Kerberos config, storage host·mount와 Kubernetes context를 제공한다.

`inventory.py` 는 두 정보를 결합해 Kerberos principal, NFS mount target, Kubernetes context와 public health port를 만든다.

5.3 명령 구성

`planner.py` 는 대상 서버와 구성요소를 결합해 inventory, playbook, host, tag와 extra vars가 포함된 Ansible 명령을 만든다.

`commands.py` 는 `audit`, `plan`, `apply` 를 실행하고 결과를 text 또는 JSON으로 출력한다.

6. 코드 위치

파일·디렉터리	역할
<code>policy/standard-gpu-server.yml</code>	구성요소와 실행 순서
<code>components/</code>	구성요소별 목표 상태, 점검·설정 진입점과 승인 수준
<code>server_state/catalog.py</code>	정책과 구성요소 로딩·검증
<code>server_state/inventory.py</code>	서버 선택과 FARM/LAB 실행값 생성
<code>server_state/planner.py</code>	Ansible 명령 생성

파일·디렉터리	역할
<code>server_state/commands.py</code>	CLI 명령과 결과 처리
<code>ansible/playbooks/audit.yml</code>	공통 구성요소 audit 순서
<code>ansible/playbooks/converge.yml</code>	공통 구성요소 설정 순서
<code>ansible/roles/</code>	구성요소별 점검·설정 task
<code>kerberos-nfs/ansible/</code>	Kerberos/NFS 점검·설정 task
<code>monitoring/ansible_playbook/</code>	exporter 점검·배포 playbook
<code>tests/</code>	정책, 서버 정보, 명령 구성과 승인 test

server-state 운영

개요 · 설계

1. 개요

이 문서의 목표는 `server-state` 를 운영하면서 서버와 구성요소를 추가하고, 환경 설정과 실행 제어를 변경하며, 서버 점검과 설정 작업을 수행하는 절차를 제공하는 것이다. 설계 문서에서 정의한 구조를 기준으로 다음 작업을 다룬다.

작업	변경 위치	확인 방법
서버 추가	<code>servers.jsonl</code> , Ansible inventory	<code>list-hosts</code> , <code>baseline-access</code> 점검
FARM/LAB 설정 변경	<code>server-state/config/environments.yml</code>	<code>plan --show-command</code> 의 extra vars 확인
구성요소 추가·수정	<code>server-state/components/</code> , Ansible role·playbook	<code>describe</code> , <code>audit</code> , <code>plan</code>
실행 형태·안전 수준 변경	구성요소 정의와 <code>server_state/</code> Python package	단위 테스트와 승인 동작 확인
서버 점검·설정	<code>bin/server-state</code> CLI	실행 결과와 종료 코드 확인

명령은 `admin_infra_server` 저장소를 clone한 루트 디렉터리(예: `/home/jy/server_manage`)에서 실행한다.

```
cd /home/jy/server_manage
./server-state/bin/server-state --help
```

2. 신규 서버 추가

신규 서버 추가는 서버 정보 등록, Ansible 접속 대상 등록, 접속 점검, 표준 구성 순서로 진행한다.

2.1 등록 정보 준비

정보	용도
<code>host</code>	CLI의 <code>--hosts</code> 에서 사용하는 소문자 서버 이름
<code>server_id</code>	Kerberos principal과 서버 식별에 사용하는 대문자 ID
<code>domain</code>	FARM 또는 LAB 환경 선택
<code>server_no</code>	공유 경로와 public health port 계산
<code>ansible_host</code> , <code>ansible_port</code> , <code>ansible_user</code>	SSH 접속
management interface-IPv4	관리 network 정보
storage interface-IPv4	storage network 설정과 점검

`host` 와 `server_id` 는 기존 서버와 겹치지 않는 값을 쓰고, `server_no` 는 같은 domain 안에서 기존 서버와 겹치지 않는 값을 쓴다. IP, hostname, SSH key와 비대화형 sudo도 이 단계에서 준비한다.

2.2 서버 정보 등록

공용 `servers.jsonl` 에 서버 한 대를 한 줄의 JSON으로 추가한다. 다음 예시는 `server-state` 가 사용하는 핵심 필드를 보여준다.

```
{"host": "farm10", "server_id": "FARM10", "domain": "FARM", "server_no": 10, "inventory":  
{"group": "FARM", "ansible_host": "192.168.2.20", "ansible_port": 8090, "ansible_user": "jy"}, "networks":  
{"management": {"name": "eno1", "ipv4": "192.168.2.20"}, "storage": {"name": "eno2", "ipv4": "100.100.100.110"}}
```

같은 서버를 `monitoring/ansible_playbook/inventory.ini` 의 FARM 또는 LAB group에도 등록한다. host 이름과 SSH 주소·port는 `servers.jsonl` 의 값과 맞춘다.

```
[FARM]  
farm10 ansible_host=192.168.2.20 ansible_port=8090
```

group과 다른 SSH 계정을 사용하는 서버는 해당 host 행에 `ansible_user` 를 함께 지정한다.

2.3 등록 결과 확인

CLI가 새 서버의 ID, domain, SSH 주소와 계산된 public health port를 읽는지 확인한다.

```
./server-state/bin/server-state list-hosts --hosts farm10
```

Ansible 접속, hostname과 sudo 조건은 `baseline-access` 점검으로 확인한다.

```
./server-state/bin/server-state audit \  
--hosts farm10 \  
--component baseline-access
```

2.4 표준 구성 진행

전체 구성요소의 예상 변경과 수동 절차를 먼저 확인한다.

```
./server-state/bin/server-state plan --hosts farm10
```

`safe`, `gated`, `risky` 순서로 설정 작업을 나누어 실행하고 각 단계가 끝날 때 동일한 구성요소를 다시 점검한다. `baseline-access`와 `kubernetes-membership`은 출력된 운영 절차에 따라 진행한다. 실행 명령과 승인 옵션은 아래의 **서버 설정** 절에서 설명한다.

3. FARM/LAB 환경 설정 변경

`server-state/config/environments.yml`은 여러 서버가 공유하는 환경 값을 관리한다.

설정	사용하는 작업
<code>kerberos_realm</code> , <code>kerberos_config_source</code>	Kerberos principal과 client 설정 구성
<code>storage_host</code> , <code>mount_source</code> , <code>mount_options</code>	NFS mount와 인증 점검
<code>kubernetes_cluster</code>	서버가 참여할 cluster 선택
<code>mount_target_template</code>	서버별 공유 경로 계산
public health port 계산값	monitoring health endpoint port 계산

환경 값을 변경할 때는 다음 순서를 사용한다.

1. FARM 또는 LAB 항목의 값을 수정한다.
2. 해당 domain의 서버 한 대를 선택해 생성될 Ansible 명령과 extra vars를 확인한다.
3. 관련 구성요소를 `audit` 하고 `plan` 으로 예상 변경을 확인한다.
4. 한 서버에 설정한 뒤 같은 domain의 나머지 서버로 범위를 넓힌다.

```
./server-state/bin/server-state plan \  
--hosts farm8 \  
--component kerberos-nfs \  
--show-command
```

public health port 계산값은 다음 명령으로 확인한다.

```
./server-state/bin/server-state list-hosts --hosts FARM
```

4. 구성요소 추가·수정

구성요소는 하나의 목표 상태와 그 상태를 확인하는 점검 작업, 목표 상태를 만드는 설정 작업을 묶는다.

4.1 점검과 설정 작업 작성

점검 작업은 서버 상태를 읽어 목표 상태 충족 여부를 판단한다. 설정 작업은 여러 번 실행해도 같은 목표 상태에 도달하도록 Ansible task를 작성한다. 공통 playbook에서 tag로 role을 선택하거나 구성요소 전용 playbook을 연결할 수 있다.

- 점검 role은 `tasks/audit.yml`에 둔다.
- 설정 role은 `tasks/main.yml`과 handler에 둔다.
- 점검 action의 `safety`는 `safe`로 지정한다.
- service 재시작, package 교체와 cluster 변경 같은 영향은 설정 action의 안전 수준에 반영한다.

4.2 구성요소 정의 추가

`server-state/components/<id>.yml`을 추가한다. 파일명과 `id`는 같아야 한다.

```
id: example-component  
desired_state: The host has the expected example service and configuration.  
audit:  
  kind: ansible-playbook  
  playbook: server-state/ansible/playbooks/audit.yml  
  tags: [example-component]  
  safety: safe  
converge:  
  kind: ansible-playbook  
  playbook: server-state/ansible/playbooks/converge.yml  
  tags: [example-component]  
  safety: gated
```

필드	작성 기준
<code>id</code>	CLI에서 선택할 고유한 이름
<code>desired_state</code>	점검과 설정이 공통으로 사용하는 목표 상태
<code>kind</code>	Ansible 실행은 <code>ansible-playbook</code> , 운영 절차 실행은 <code>manual</code>
<code>playbook</code>	저장소 루트 기준 playbook 경로
<code>tags</code>	playbook에서 이 구성요소의 task를 선택할 tag
<code>host_variable</code>	playbook이 대상 host를 받는 변수 이름. 기본값은 <code>server_state_hosts</code>
<code>reference</code>	<code>manual</code> 작업에서 표시할 운영 절차
<code>safety</code>	<code>safe</code> , <code>gated</code> , <code>risky</code> 중 설정 영향에 맞는 수준

4.3 정책 순서에 추가

전체 서버가 사용하는 구성요소는 `server-state/policy/standard-gpu-server.yml`의 `components`에 추가한다. 필요한 package, runtime, network와 인증 조건을 기준으로 선행 구성요소 뒤에 배치한다. `describe`, `audit`, `plan`과 `apply`는 이 순서를 사용한다.

4.4 기존 구성요소 수정

변경 내용	수정 위치
목표 상태 문구	<code>components/<id>.yml</code> 의 <code>desired_state</code>
점검 기준	<code>audit role·playbook</code>
서버 설정 내용	<code>converge role·playbook</code> 과 <code>handler</code>
<code>playbook·tag·실행 형태·안전 수준</code>	<code>components/<id>.yml</code> 의 <code>audit</code> , <code>converge</code>
구성요소 실행 순서	<code>policy/standard-gpu-server.yml</code>

변경 후에는 한 서버에서 `audit`, `plan`, `apply`, `audit` 순서로 결과를 확인한 뒤 domain 단위로 실행 범위를 넓힌다.

4.5 연결 확인

```
./server-state/bin/server-state describe \
--component example-component

./server-state/bin/server-state audit \
--hosts farm8 \
--component example-component \
--show-command

./server-state/bin/server-state plan \
--hosts farm8 \
--component example-component \
--show-command
```

5. 실행 형태와 안전 수준 변경

설정 작업의 실행 제어는 `converge.kind` 와 `converge.safety` 로 나뉜다.

5.1 실행 형태

kind	동작
ansible-playbook	plan 에서 check/diff를 실행하고 apply 에서 설정 playbook을 실행한다.
manual	CLI가 reference 에 기록된 운영 절차를 표시한다.

manual 구성요소도 작업 영향을 나타내는 safety 를 기록한다. apply 는 해당 구성요소를 MANUAL 로 표시하며(상태값 설명은 9절 참고) 관리자가 reference의 절차를 수행한다.

5.2 안전 수준

safety	설정 실행 조건
safe	apply --execute
gated	apply --execute --approve-gated
risky	apply --execute --approve-risky

audit 은 읽기 전용 작업이므로 항상 safe 를 사용한다. 기존 구성요소의 실행 형태나 안전 수준을 바꿀 때는 components/<id>.yaml 의 converge 를 수정하고 describe , plan 과 승인 동작을 확인한다.

5.3 새 실행 형태 추가

새 kind 는 YAML 값과 해당 값을 처리하는 실행 코드를 함께 추가한다.

1. `catalog.py`의 `ACTION_KINDS`, `Action` 필드와 로딩 검증을 확장한다.
2. `planner.py`에서 새 실행 형태에 필요한 `PlanItem`을 만든다.
3. `commands.py`에서 실행 결과와 오류를 표시한다.
4. `catalog`, `planner`와 `command` 단위 테스트에 로딩·계획·실행 사례를 추가한다.
5. 구성요소 하나에 새 값을 연결해 `describe`, `plan`, `apply`를 확인한다.

5.4 새 안전 수준 추가

새 `safety`는 승인 조건과 CLI 옵션을 함께 설계한다.

1. `catalog.py`의 `SAFETY_LEVELS`와 검증 사례를 확장한다.
2. `commands.py`의 `apply` option과 승인 판정을 추가한다.
3. 승인 전 `BLOCKED`, 승인 후 실행되는 `command test`를 추가한다.
4. 설계 문서와 운영 문서에 영향 범위와 실행 명령을 기록한다.
5. 한 서버에서 차단·승인·실행·재점검 흐름을 확인한다.

6. 대상과 정책 확인

관리 대상과 파생된 `public health port`를 확인한다.

```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

`--hosts`는 `all`, `FARM/LAB domain`, `host 이름`, `server ID`와 쉼표·공백으로 구분한 조합을 지원한다.

전체 정책이나 선택한 구성요소의 목표 상태, 점검·설정 실행 형태와 안전 수준을 확인한다.

```
./server-state/bin/server-state describe
./server-state/bin/server-state describe --component docker-engine
./server-state/bin/server-state describe \
  --component docker-engine,kerberos-nfs,monitoring
```

`--component`를 생략하면 전체 정책을 선택한다. 여러 번 사용하거나 쉼표로 묶을 수 있으며, 실행 순서는 정책의 구성요소 순서를 따른다.

7. 운영 서버 점검

7.1 실행 명령 확인

`--show-command`는 실제로 사용할 Ansible inventory, playbook, tag와 extra vars가 포함된 명령을 화면에 출력한다.

```
./server-state/bin/server-state audit \  
--hosts farm8 \  
--component docker-engine \  
--show-command
```

7.2 점검 실행

`--show-command` 를 제외하면 선택한 서버에 접속해 읽기 전용 audit task를 실행한다.

```
# 한 서버의 전체 구성요소 점검  
./server-state/bin/server-state audit --hosts farm8  
  
# FARM 전체의 Docker와 NVIDIA runtime 점검  
./server-state/bin/server-state audit \  
--hosts farm \  
--component docker-engine \  
--component nvidia-runtime
```

각 구성요소는 별도 Ansible 실행으로 처리된다. 결과에는 서버와 구성요소가 함께 표시되며 하나 이상의 실행이 실패하면 종료 코드는 1이다.

8. 서버 설정

8.1 예상 변경 확인

`plan` 은 설정 playbook을 Ansible `--check --diff` 로 실행한다. package, 파일과 service의 예상 변경을 확인하며, `manual` 구성요소는 운영 절차 reference를 표시한다.

```
./server-state/bin/server-state plan \  
--hosts farm8 \  
--component os-common \  
--component docker-engine \  
--component storage-network
```

명령 구성만 확인할 때는 `--show-command` 를 추가한다.

8.2 `safe` 설정 실행

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component os-common \  
--component storage-network \  
--execute
```

8.3 **gated** 설정 승인과 실행

예상 diff와 service 재시작, package·설정 변경 영향을 확인한 뒤 승인한다.

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component docker-engine \  
--component nvidia-runtime \  
--component kubernetes-packages \  
--execute \  
--approve-gated
```

8.4 **risky** 설정 승인과 실행

GPU workload와 reboot 일정까지 확인한 뒤 승인한다.

```
./server-state/bin/server-state apply \  
--hosts farm8 \  
--component nvidia-driver \  
--execute \  
--approve-risky
```

`--approve-risky` 는 `gated` 승인도 포함한다. 필요한 승인이 빠지면 Ansible 실행 전에 종료 코드 2로 끝난다. `manual` 구성요소는 Ansible로 실행할 수 없으므로 `apply` 요청에 포함돼도 같은 방식으로 종료 코드 2가 된다. 검토가 끝난 구성요소를 안전 수준별로 선택해 실행한다.

8.5 설정 후 점검

설정에 사용한 서버와 구성요소를 같은 조건으로 다시 점검한다.

```
./server-state/bin/server-state audit \  
--hosts farm8 \  
--component docker-engine \  
--component nvidia-runtime
```

9. 결과 해석

상태	의미
COMMAND	<code>--show-command</code> 로 실행 예정 명령을 출력했다.
OK	해당 Ansible 실행이 종료 코드 0으로 끝났다.
FAILED	Ansible 실행이 실패했다. <code>detail</code> 의 stdout·stderr를 확인한다.
MANUAL	표시된 운영 절차 reference를 사용한다.
BLOCKED	선택한 안전 수준에 필요한 승인을 추가한다.

자동 처리용 JSON은 전역 옵션인 `--format` 을 하위 명령 앞에 둔다.

```
./server-state/bin/server-state --format json audit \  
  --hosts farm8 \  
  --component docker-engine \  
  --show-command
```

종료 코드는 전체 성공 0, Ansible 실행 실패 1, 설정 오류 또는 승인 차단 2다.

10. 변경 검증

Python 단위 테스트:

```
cd /home/jy/server_manage/server-state  
python3 -m unittest discover -s tests -v
```

주요 Ansible 구문 검사:

```
cd /home/jy/server_manage
ANSIBLE_CONFIG="$PWD/monitoring/ansible_playbook/ansible.cfg" \
ANSIBLE_ROLES_PATH="$PWD/server-state/ansible/roles:$PWD/kerberos-nfs/ansible/roles" \
ansible-playbook --syntax-check \
  -i monitoring/ansible_playbook/inventory.ini \
  server-state/ansible/playbooks/audit.yml

ANSIBLE_CONFIG="$PWD/monitoring/ansible_playbook/ansible.cfg" \
ANSIBLE_ROLES_PATH="$PWD/server-state/ansible/roles:$PWD/kerberos-nfs/ansible/roles" \
ansible-playbook --syntax-check \
  -i monitoring/ansible_playbook/inventory.ini \
  server-state/ansible/playbooks/converge.yml
```

CLI 연결 확인:

```
./server-state/bin/server-state audit \
  --hosts farm8 \
  --component docker-engine \
  --show-command

./server-state/bin/server-state plan \
  --hosts lab10 \
  --component kerberos-nfs \
  --show-command
```

운영 반영은 한 서버에서 **audit**, **plan**, **apply**, **audit** 순서로 검증한 뒤 같은 domain의 서버로 범위를 넓힌다.